INTERNATIONAL CYBERSECURITY & DIGITAL FORENSICS ACADEMY (ICDFA)

CIP-B101 - Basic Computer Skills for Digital Forensics

Lab 2A: Linux Command Line for Digital Forensics Basics

Full Name	Nebeuwa Ifeanyichukwu Raphael
Student ID	202520820LE
Course Code	CIP-B101
Course Title	Basic Computer Skills for Digital Forensics
Lab Code	Lab 2A
Lab Title	Linux Command Line for Digital Forensics Basics
Instructor Name	Ammar Aminu
Date of Submission	16/08/2026

Executive Summary

This report documents the completion of Lab 2A, a practical exercise in using the Linux command line for foundational digital forensic tasks. The exercise required establishing a structured evidence workspace, examining the Linux filesystem hierarchy and core operating system information, inspecting inode and disk metadata, navigating using absolute and relative paths, performing safe file management operations, searching evidence content with grep, locating files with tree and find, reviewing network configuration through authorized localhost-only testing, writing and executing a shell script, making that script callable from any location via the PATH variable, and finally compressing the completed workspace for submission. The overall conclusion drawn from this lab is that the Linux command line, approached methodically, provides a complete and auditable toolkit for early-stage evidence handling — from locating and inspecting files at the metadata level through to packaging a finished body of work for submission or handover.

Lab Objectives

On completion of this lab, the following objectives were expected to be met:

- Explain common operating system functions and why Linux is useful in digital forensics.
- Identify the Linux filesystem structure and explain the idea that "everything is a file".
- Use inode and disk-free commands to inspect file and filesystem information.
- Navigate Linux directories using absolute and relative paths.
- Create, copy, rename, delete, search, compress, and extract files using command-line tools.
- Use grep to search evidence text and document results.
- Check basic network information and test only authorized local connectivity.
- Create and execute a simple shell script, then make it callable using the PATH variable.

Tools Used / Environment

Item	Detail
Operating System	Kali Linux, Ubuntu, Debian, or another Linux distribution approved by the instructor
User Access	Normal user account with sudo access for installation-only tasks
Tools	Terminal/Terminator, tree, grep, nano, ss/netstat, ping, nmap, wget, zip/unzip, tar
Internet	Optional - required only for installing missing packages or visiting reference links
Evidence Workspace	A folder created under the home directory: ~/CIP-B101_Lab2A_FIRSTNAME_LASTNAME
Package installation (if required)	sudo apt update && sudo apt install -y tree nmap wget zip unzip tar net-tools terminator nano

Forensic safety note

All commands in this lab were executed only on the student's own Linux/Kali VM or an authorized classroom lab system. No scanning, probing, or connection to external systems was performed, other than the localhost and public reference material explicitly permitted by the lab brief.

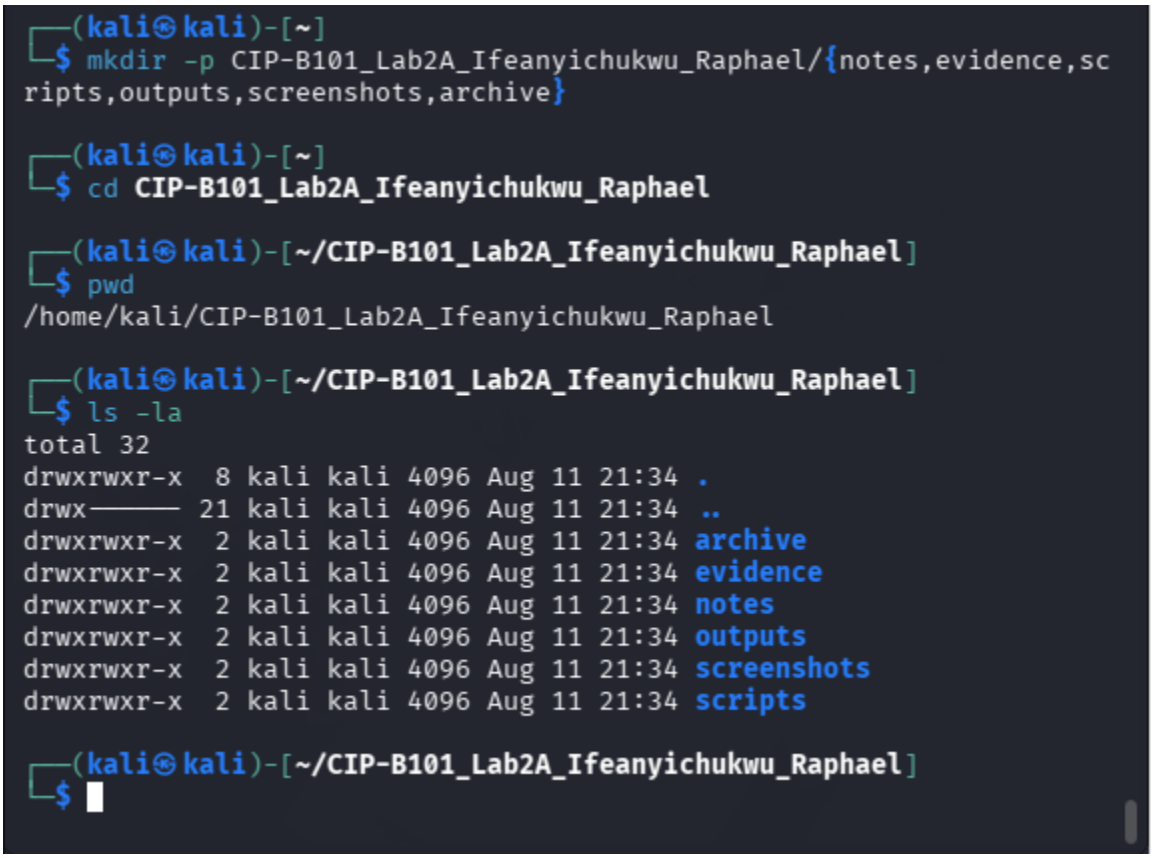
Methodology

Work was carried out entirely through the Linux terminal on the designated training VM, following the eleven-task sequence set out in the lab brief. A dedicated workspace was created first, with subfolders separating notes, evidence, scripts, outputs, screenshots and the final archive, so that every artefact produced during the lab remained isolated and traceable. Each task built on native Linux utilities file system navigation and inspection commands, inode and disk-usage tools, grep-based text search, tree and find for file discovery, basic networking commands restricted to localhost, and simple shell scripting — reflecting the same category of tools a forensic analyst would rely on during early

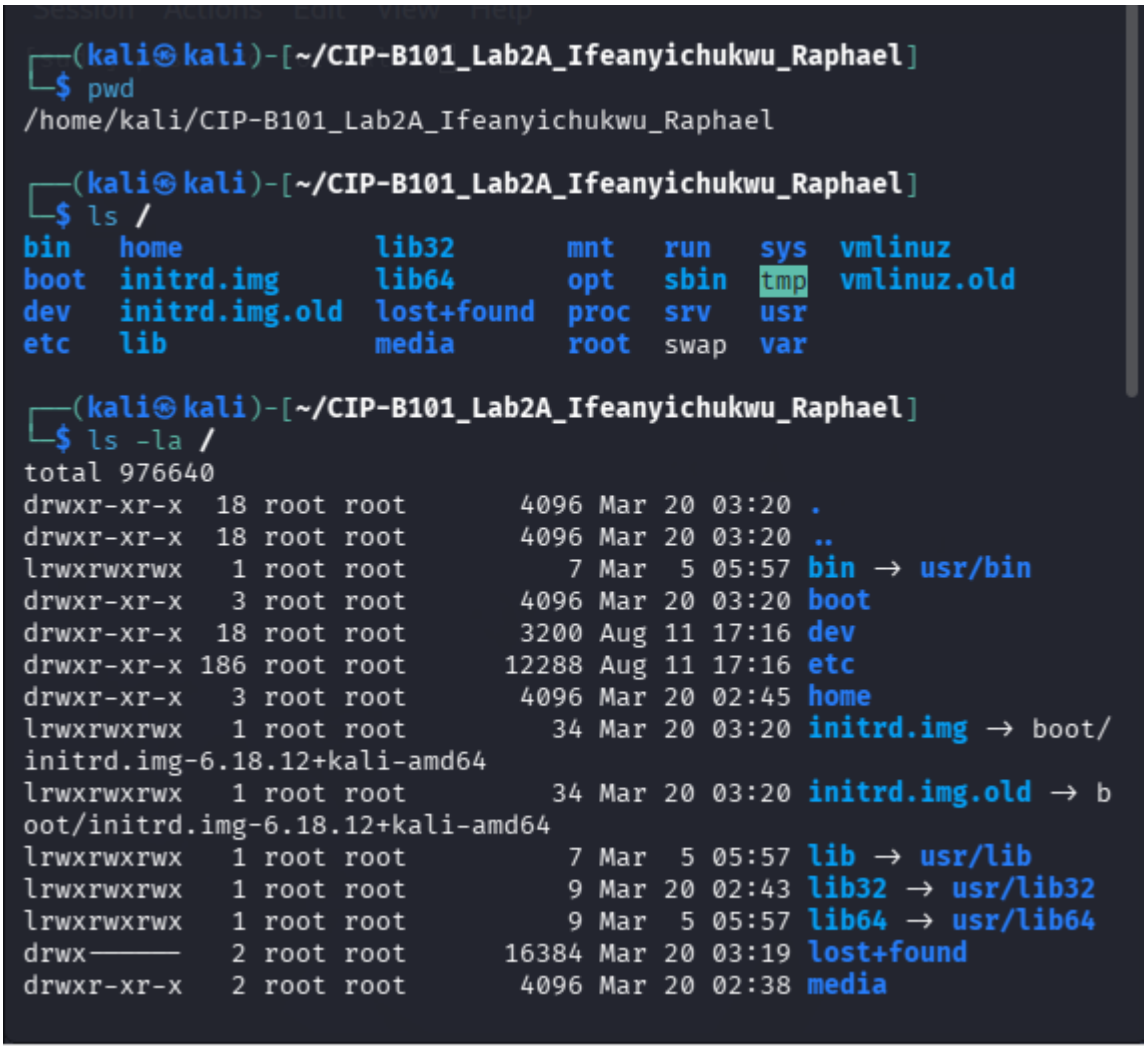
triage of a Linux host. The completed workspace was compressed at the end of the lab to produce a single submittable evidence package.

Task 1 - Prepare Your Evidence Workspace

A structured workspace was created under the home directory to hold all commands, outputs, screenshots and final evidence artefacts for this lab.



Task 2 - Inspected Linux File system and OS Information



```
┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.1"
VERSION="2026.1"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"

┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ uname -a
Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1k
ali1 (2026-02-25) x86_64 GNU/Linux

┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ whoami
kali

┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),2
4(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev)
,100(users),101(netdev),102(scanner),104(bluetooth),113(lpadmin),122
(wireshark),123(kaboxer)
```

2.1 Explanation of key top-level directories

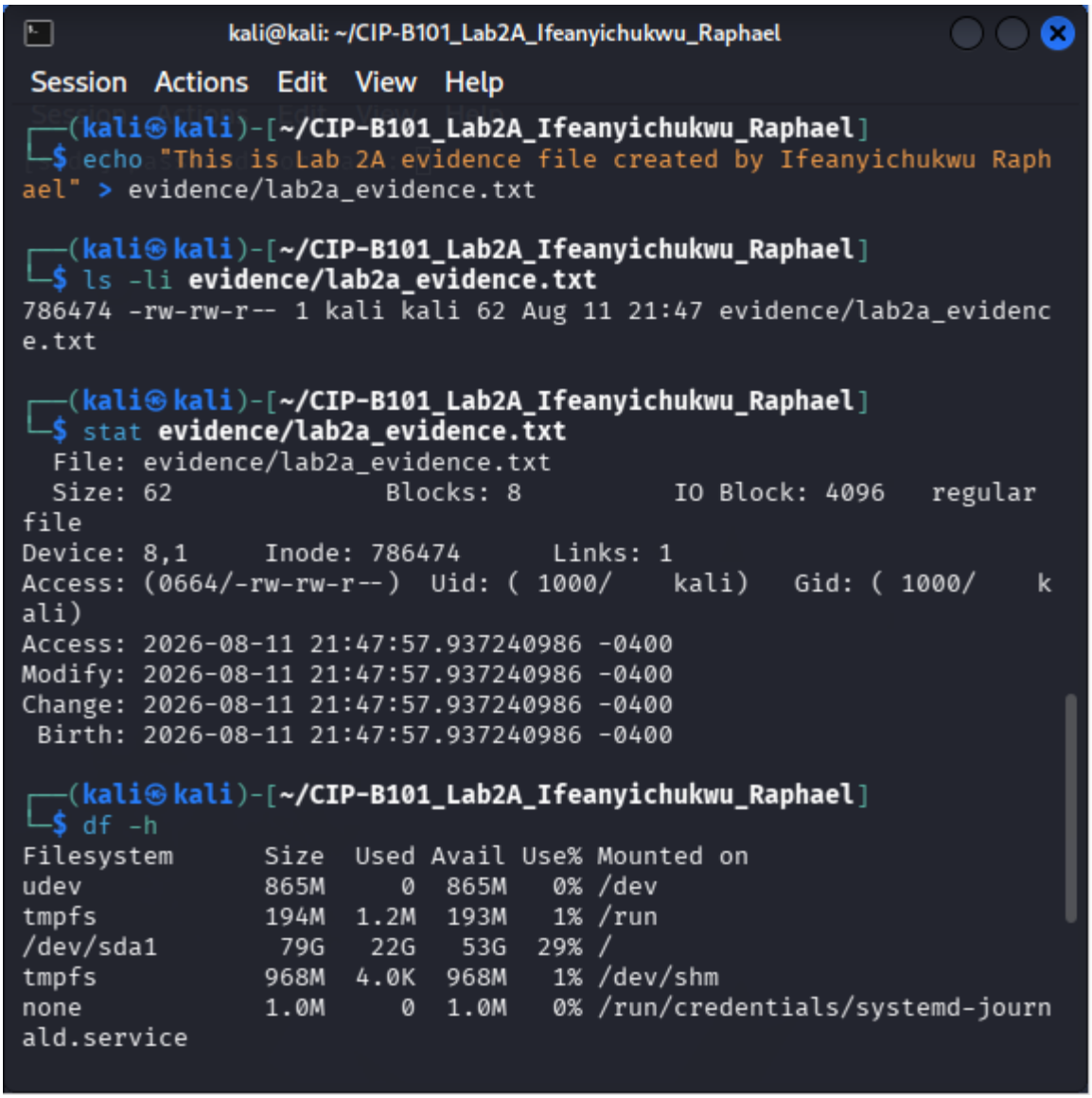
Directory	Purpose
/home	Contains personal directories for each regular user, where user-created files and evidence copies are typically stored
/etc	Holds system-wide configuration files, such as network settings and installed service configuration
/proc	A virtual filesystem exposing live kernel and process information; its contents exist only in memory and disappear on reboot
/sys	A virtual filesystem exposing kernel and hardware device information, used by the system to manage device drivers and settings

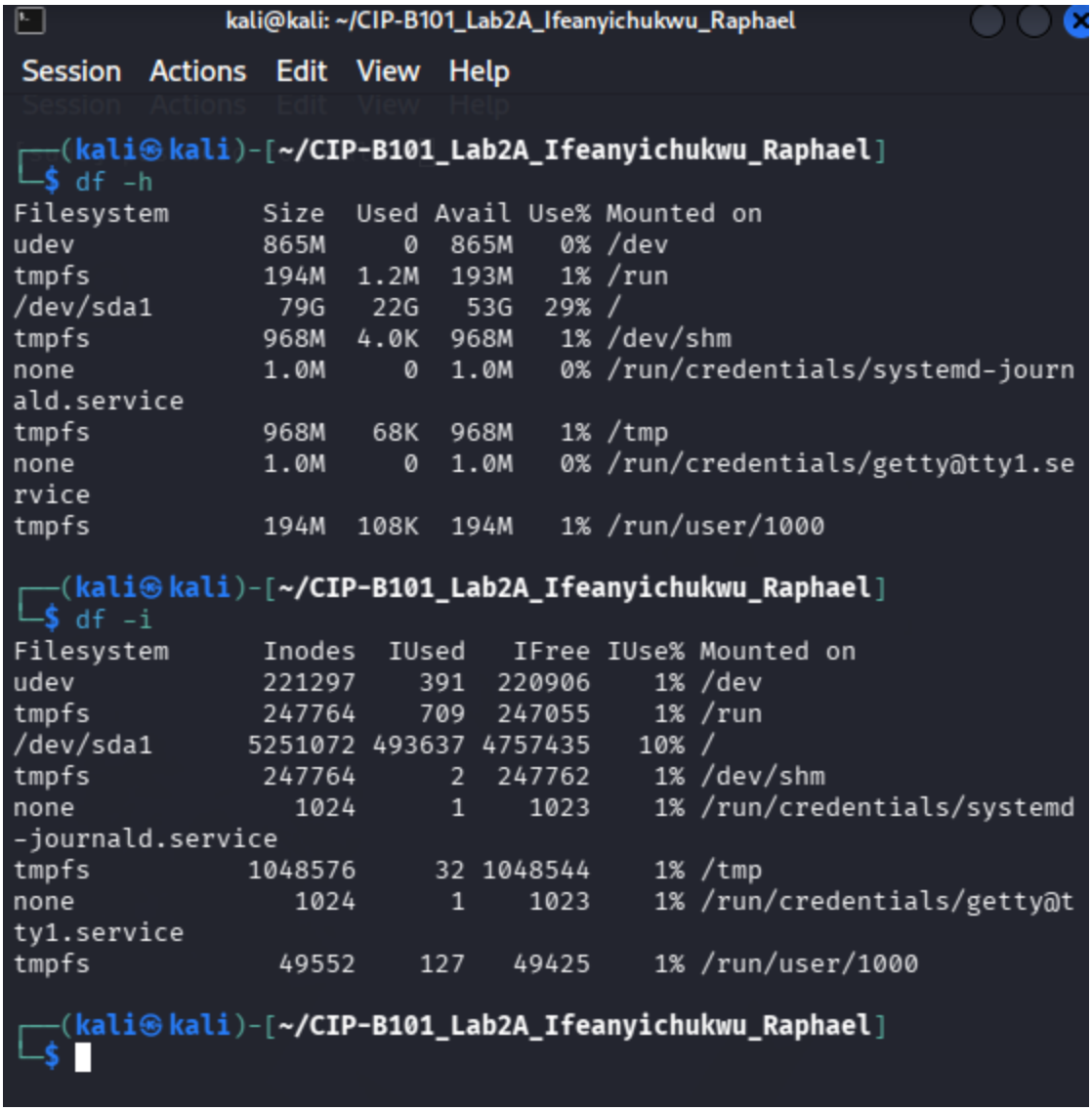
/dev	Contains device files representing hardware and virtual devices, consistent with the Linux principle that hardware is also represented as a file
/bin	Contains essential command-line executables required for basic system operation, available to all users
/tmp	A temporary storage location for files needed only during the current session; contents are typically cleared on reboot

From a forensic standpoint, this distinction matters because /proc and /sys are volatile and reflect only the live state of the system at the moment they are queried, whereas /etc, /home and /bin persist on disk and can be examined after the fact. Evidence gathered from /proc must therefore be captured while the system is running, since it cannot be recovered afterward in the same way a file in /home can.

Task 3 - Understand Inodes and Disk Information

An evidence file, lab2a_evidence.txt, was created inside the evidence folder using echo, and its inode number was retrieved using ls -li, with stat used afterward to view the file's full metadata, including permissions, ownership, and timestamps. df -h was then run to check available disk space across mounted filesystems, and df -i was run to check inode usage, since a filesystem can run out of usable inodes even when disk space itself is not full, particularly on systems holding a very large number of small files. The screenshot below captures the inode number assigned to the evidence file, its full stat output, and the corresponding df -h and df -i results for the system.


kali@kali: ~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael
Session Actions Edit View Help
┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ echo "This is Lab 2A evidence file created by Ifeanyichukwu Raphael" > evidence/lab2a_evidence.txt
┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ ls -li evidence/lab2a_evidence.txt
786474 -rw-rw-r-- 1 kali kali 62 Aug 11 21:47 evidence/lab2a_evidence.txt
┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ stat evidence/lab2a_evidence.txt
File: evidence/lab2a_evidence.txt
Size: 62 Blocks: 8 IO Block: 4096 regular file
Device: 8,1 Inode: 786474 Links: 1
Access: (0664/-rw-rw-r--) Uid: (1000/ kali) Gid: (1000/ kali)
Access: 2026-08-11 21:47:57.937240986 -0400
Modify: 2026-08-11 21:47:57.937240986 -0400
Change: 2026-08-11 21:47:57.937240986 -0400
Birth: 2026-08-11 21:47:57.937240986 -0400
┌──(kali㉿kali)-[~/CIP-B101_Lab2A_Ifeanyichukwu_Raphael]
└─$ df -h
Filesystem Size Used Avail Use% Mounted on
udev 865M 0 865M 0% /dev
tmpfs 194M 1.2M 193M 1% /run
/dev/sda1 79G 22G 53G 29% /
tmpfs 968M 4.0K 968M 1% /dev/shm
none 1.0M 0 1.0M 0% /run/credentials/systemd-journald.service



What an inode is and why it matters forensically

An inode is a metadata structure that Linux uses to represent a file on disk. It stores information such as file size, permissions, ownership, timestamps, and pointers to the actual data blocks, but it does not store the file's name - the name is only a label held in the containing directory, which points to the inode. This matters in digital forensics because a file can be renamed, or even have multiple names (hard links) pointing to the same inode, without the underlying data or its metadata changing at all. Conversely, when a file is deleted, its directory entry is typically removed while the inode and its data blocks may remain temporarily

intact on disk until they are overwritten, which is part of why deleted files can sometimes still be recovered. Examining inode numbers and stat output therefore allows an examiner to work with a file's true underlying identity and history, rather than relying solely on whatever name currently happens to be associated with it.

Task 4 - Absolute and Relative Paths

Navigation was tested by moving between the evidence and notes subfolders using both absolute and relative references, with pwd run after each move to confirm the actual working directory. cd .. was used to move up one level relative to the current location, cd ./notes to move into a named subfolder relative to the current position, and cd ../evidence to move back down into a sibling folder — each of these only resolves correctly from a known starting point, unlike a full absolute path, which remains valid regardless of the current directory. readlink -f was also used on the evidence file to resolve and display its complete absolute path directly. The screenshot below shows the pwd output confirming each navigation step, along with the readlink -f result.



Absolute path example

/home/FIRSTNAME/CIP-B101_Lab2A_FIRSTNAME_LASTNAME/evidence/lab2a_evidence.txt - this identifies the file uniquely from the file system root, regardless of the current working directory.

Relative path example

../evidence/lab2a_evidence.txt - this only resolves correctly when run from a known location, such as the notes folder; the same relative expression would point somewhere else entirely if run from a different directory.

Task 5 - Create, View, Copy, Rename and Remove Files

Two case note files were created in the notes folder using echo and viewed with cat to confirm their content. cp was then used to copy one note into the evidence folder, and cp -r was used separately to copy the entire notes folder recursively, including its contents, into evidence as a backup. mv was used to rename the copied file within evidence without moving it elsewhere, and a clearly labelled temporary test file was created and then permanently removed using rm, to demonstrate deletion without risking any actual evidence file. The screenshot below shows the evidence folder listing before and after these operations, confirming the copy, rename, and successful deletion of only the temporary file.

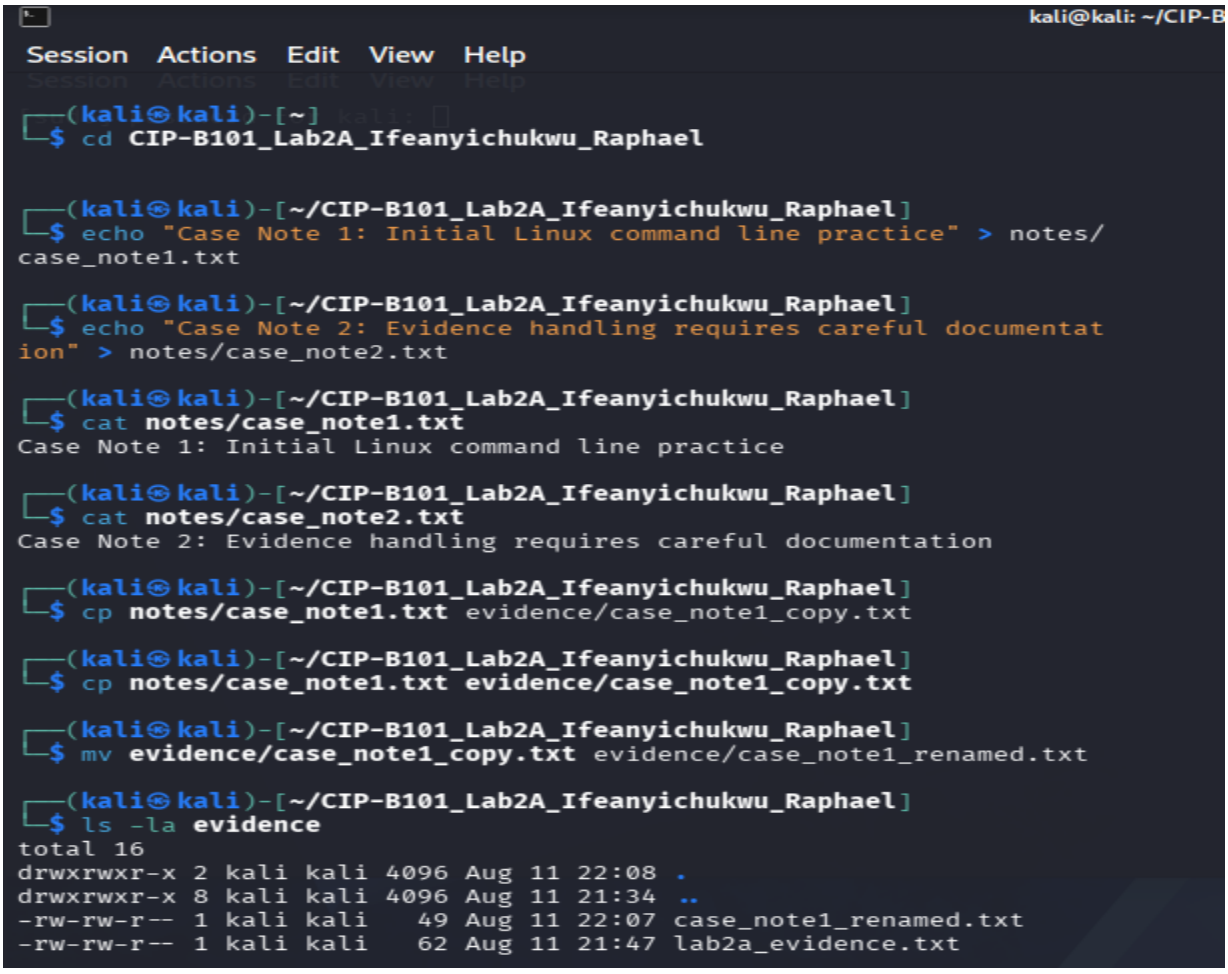
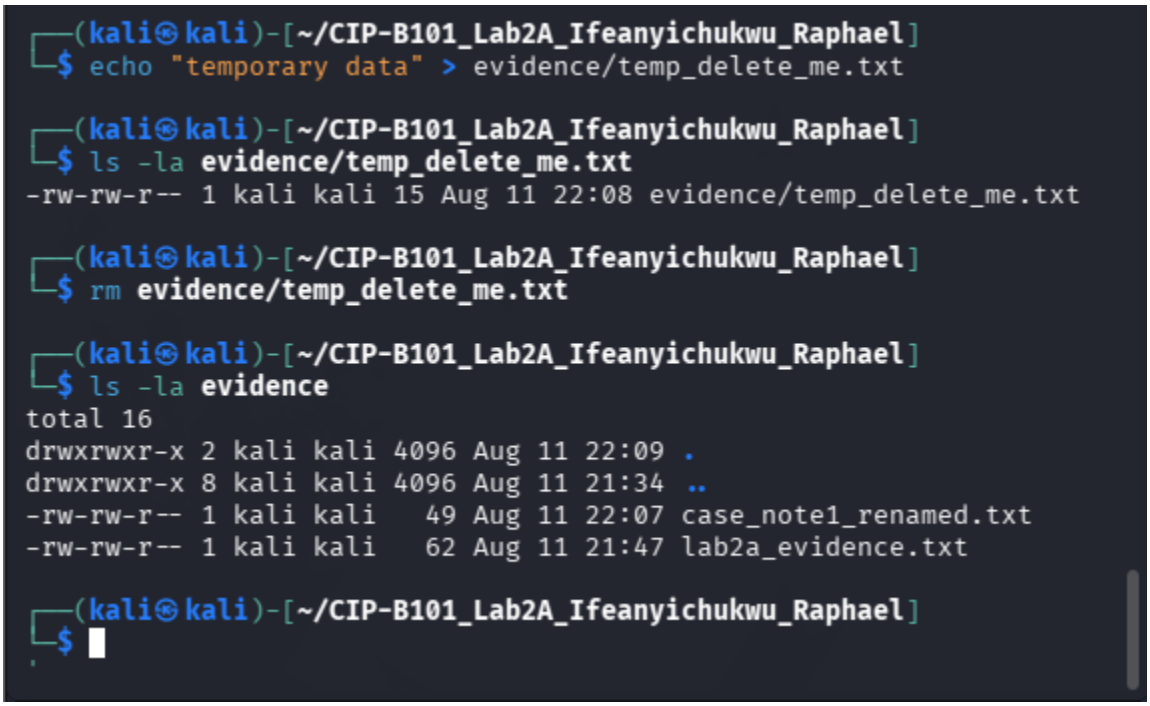


Fig 5.1 Created and removing a temporary file

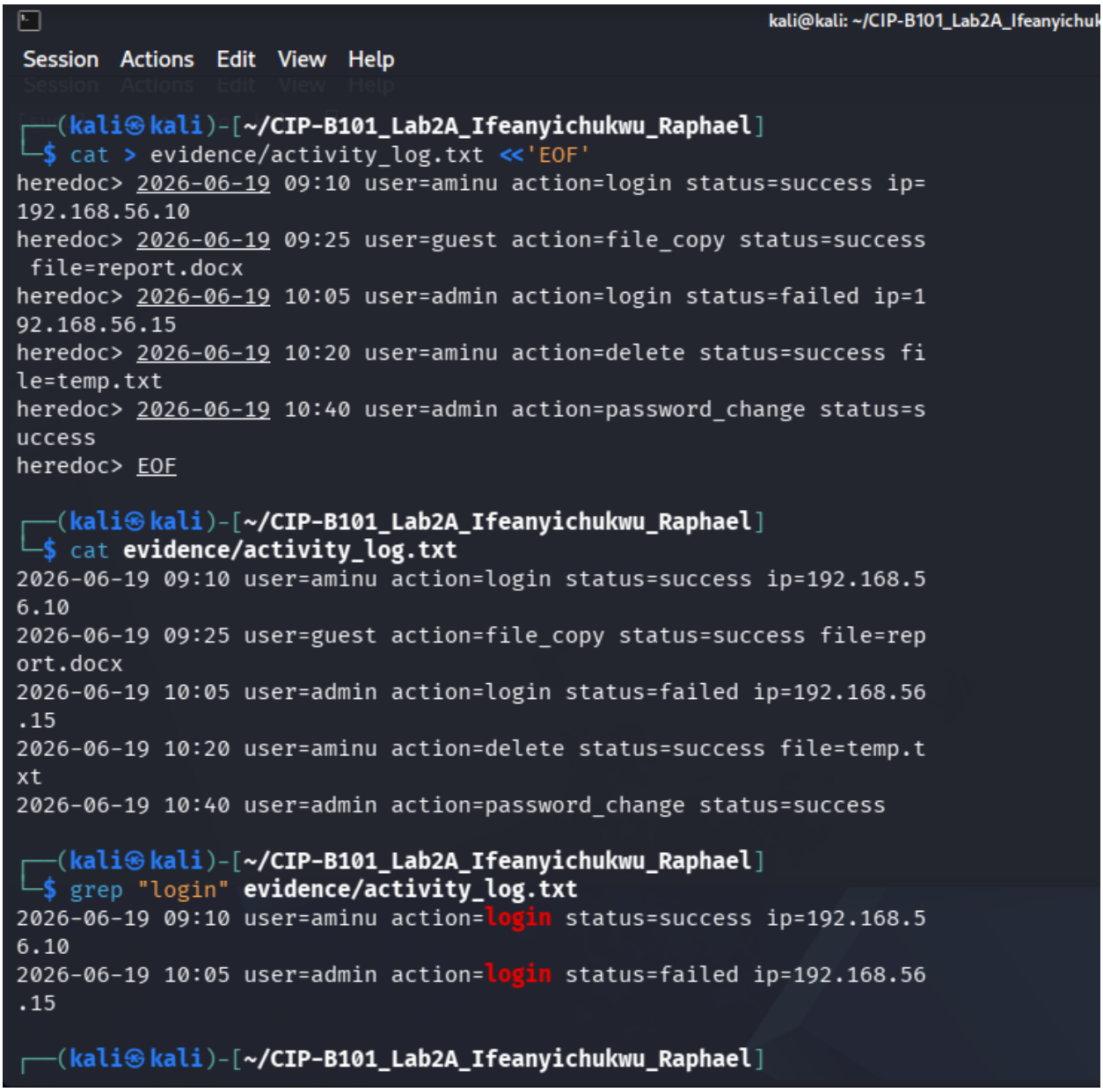


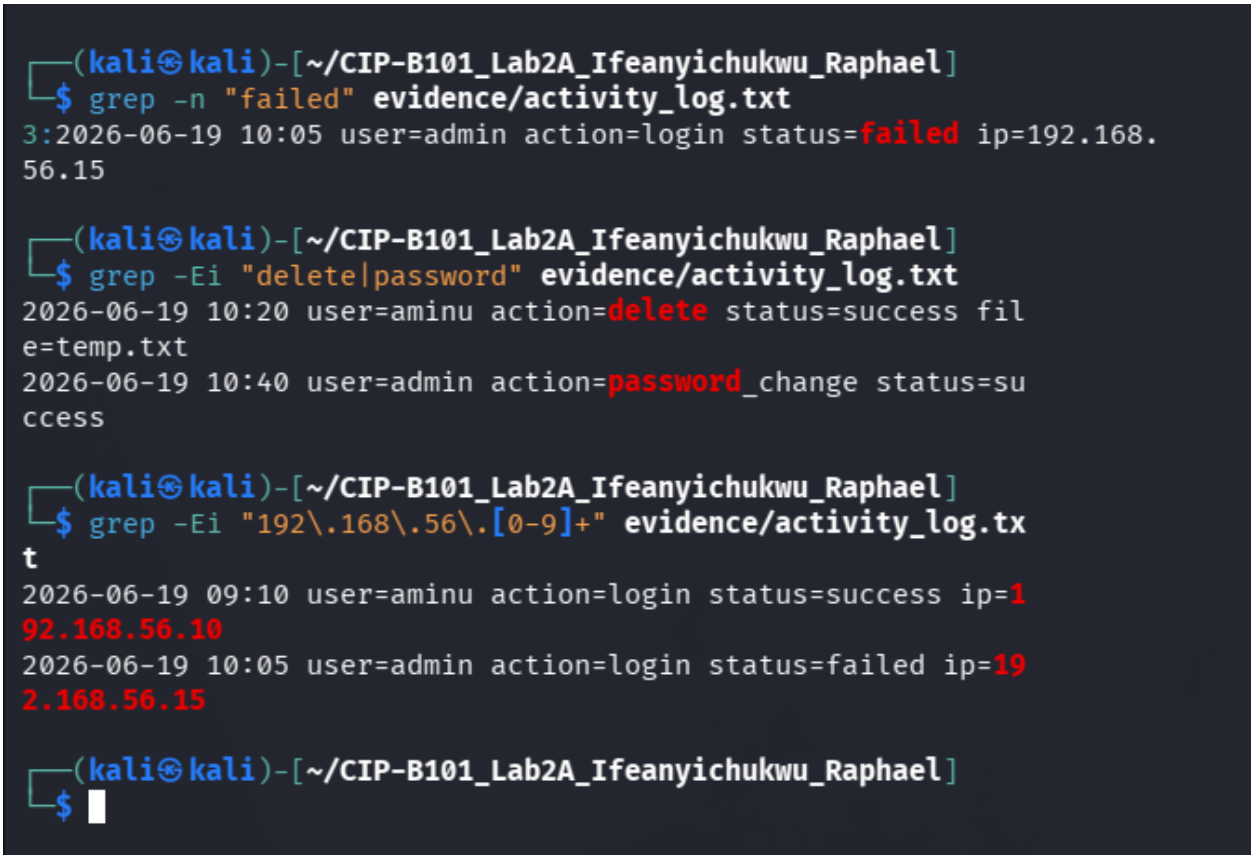
cp copies a file while leaving the original untouched; the -r flag extends this to copy an entire directory recursively, including its contents. My moves or renames a file in place, which in this task was used to rename a copied file rather than move it elsewhere. rm permanently removes a file from the directory listing; unlike a graphical file manager, the standard Linux rm command does not send files to a recoverable recycle bin by default, so this operation was deliberately limited to a clearly labelled temporary test file rather than any evidence file.

Task 6 - Searched Evidence Content with grep

A sample activity log, activity_log.txt, was created inside evidence using a heredoc, containing five entries representing login attempts, a file action, a deletion, and a password change, each with a timestamp, user, action, status, and in some cases an IP address. grep was then used to search this log for specific indicators: a plain search for "login," a line-numbered search for "failed" using the -n flag, a case-insensitive combined search for "delete" or "password" using -Ei, and a regular-expression search matching any IP address in the 192.168.56.x

range. The screenshot below shows the full log content followed by the result of each of these four search commands.





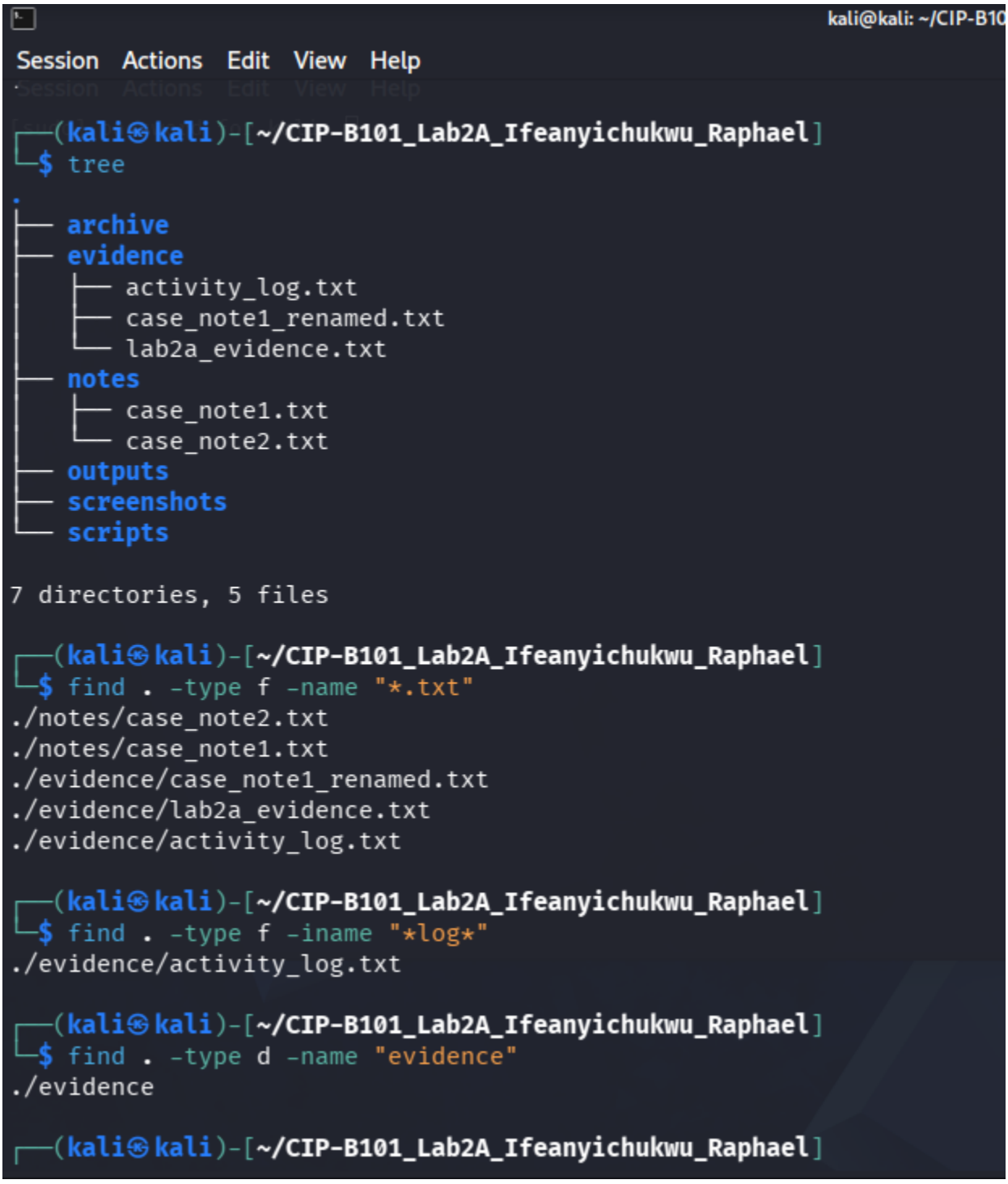
Command showing failed login evidence

grep -n "failed" evidence/activity_log.txt identifies the failed login evidence directly, returning the 10:05 entry for user=admin with status=failed. The -n flag also prints the matching line number, which allows the exact location of the match within the file to be referenced precisely rather than only confirming that a match exists somewhere in the log.

Command showing suspicious account activity

grep -Ei "delete|password" evidence/activity_log.txt surfaces potentially sensitive account activity by matching either a delete action or a password change in a single case-insensitive search, returning both the 10:20 delete entry and the 10:40 password_change entry. The final command, grep -Ei "192\.168\.56\.[0-9]+" evidence/activity_log.txt, uses a basic regular expression to match any IP address in the 192.168.56.x range, which is useful for isolating all activity originating from a specific subnet of interest during an investigation.

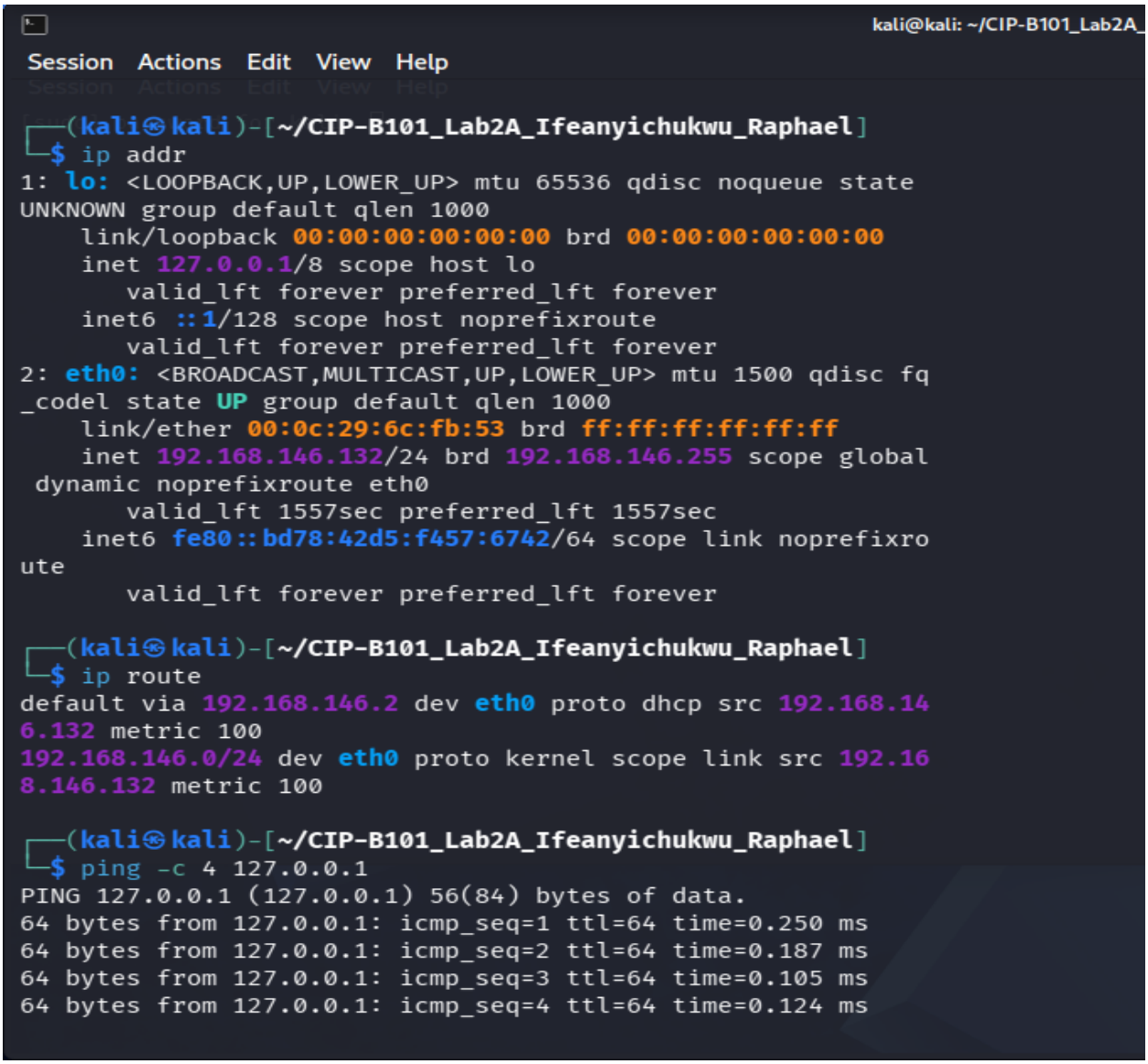
Task 7 - Searching File Names and Display Directory Tree

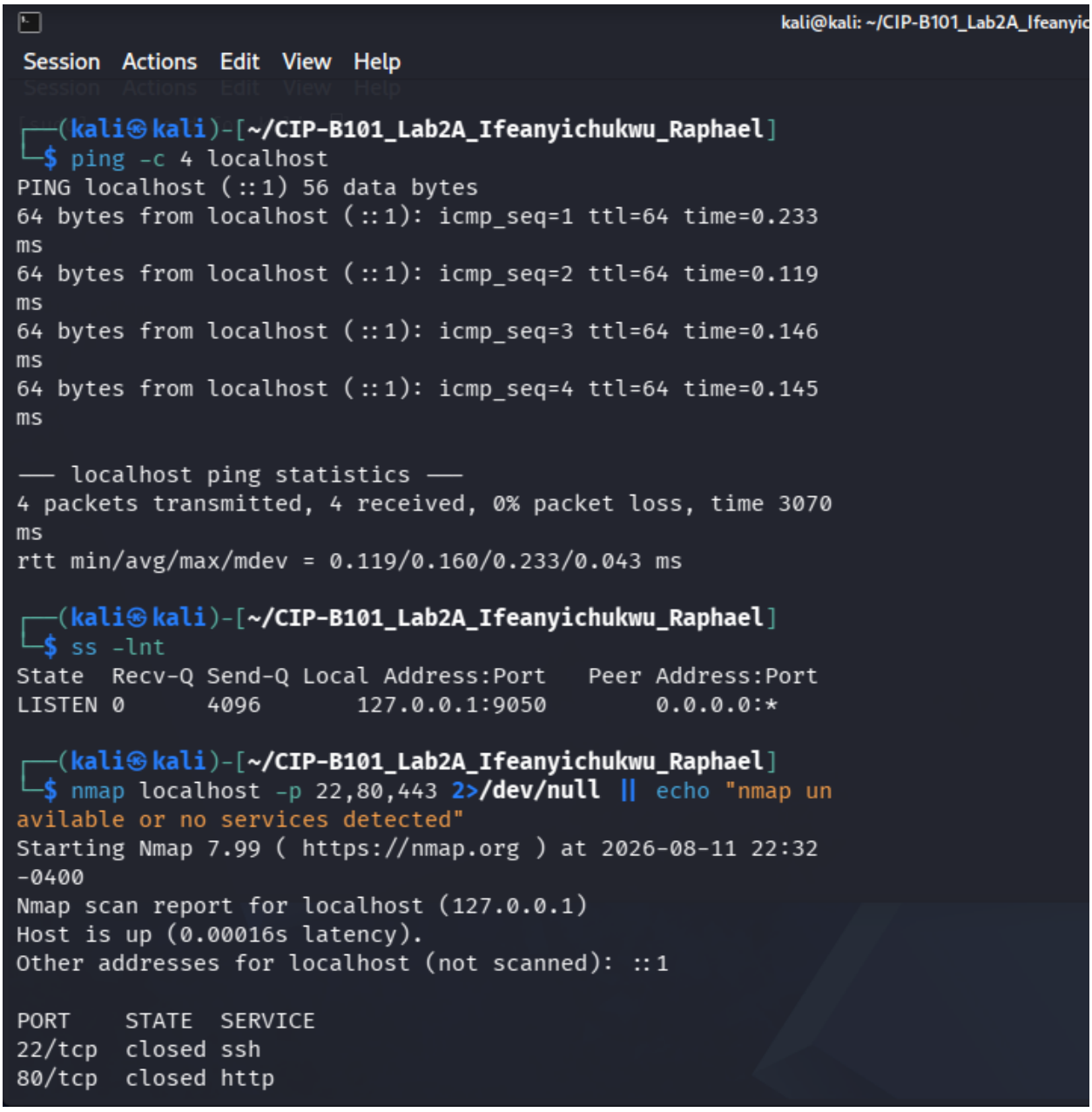


tree provides a visual map of the entire workspace hierarchy in one view, which is useful for quickly confirming overall folder structure. find, by contrast, searches

by specific criteria -type f restricts results to regular files, -name performs a case-sensitive filename match, and -iname performs the same match case-insensitively which makes it better suited to locating a specific file or category of file (such as every .txt file, or anything with "log" in its name) within a large or unfamiliar directory tree.

Task 8 – Checking Network Information Safely





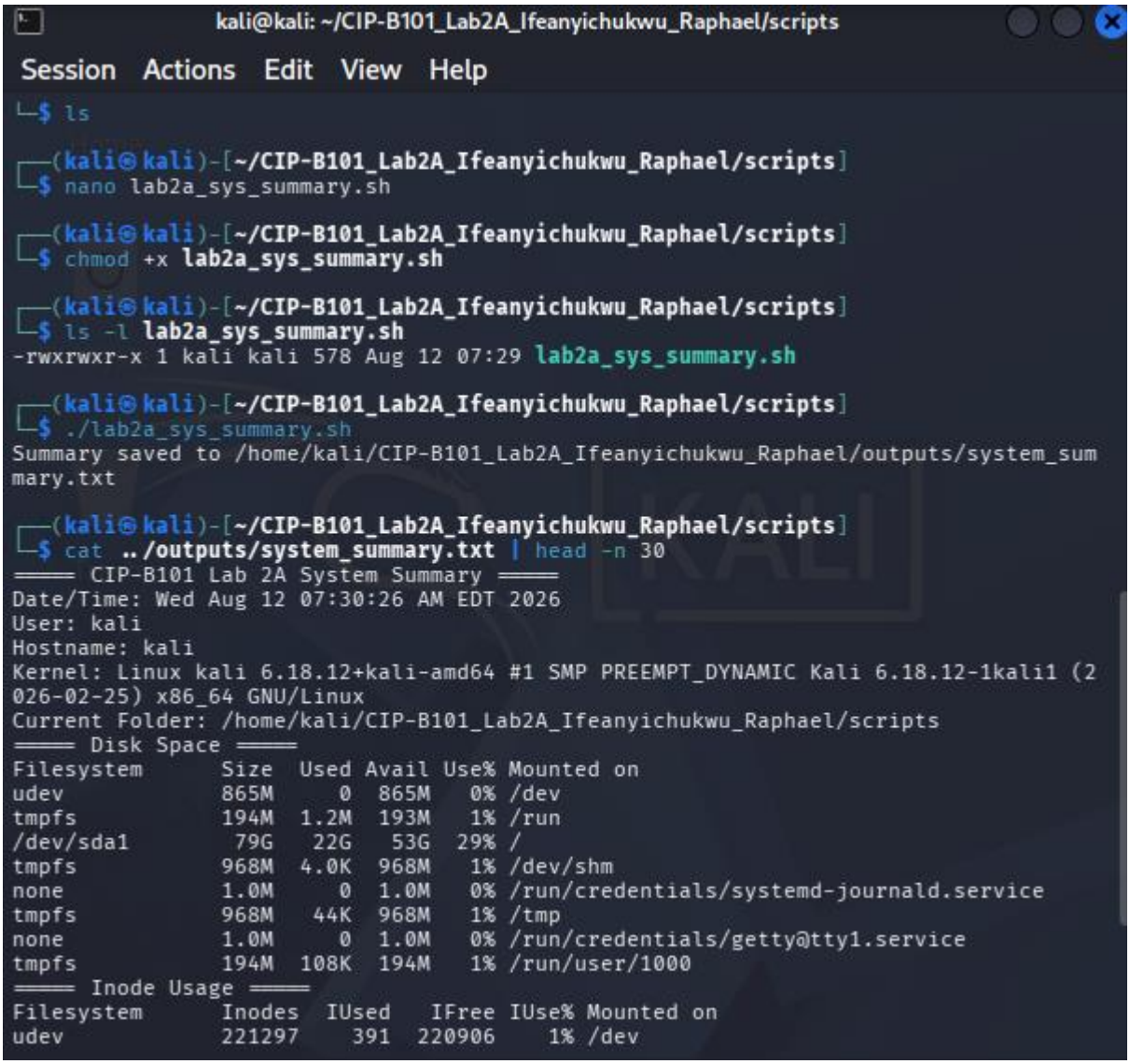
8.1 Loopback address

127.0.0.1 (also reachable via the hostname localhost) — this is the standard IPv4 loopback address, which always routes back to the local machine itself rather than out over any physical or virtual network interface.

8.2 Why localhost testing is safer than scanning unknown systems

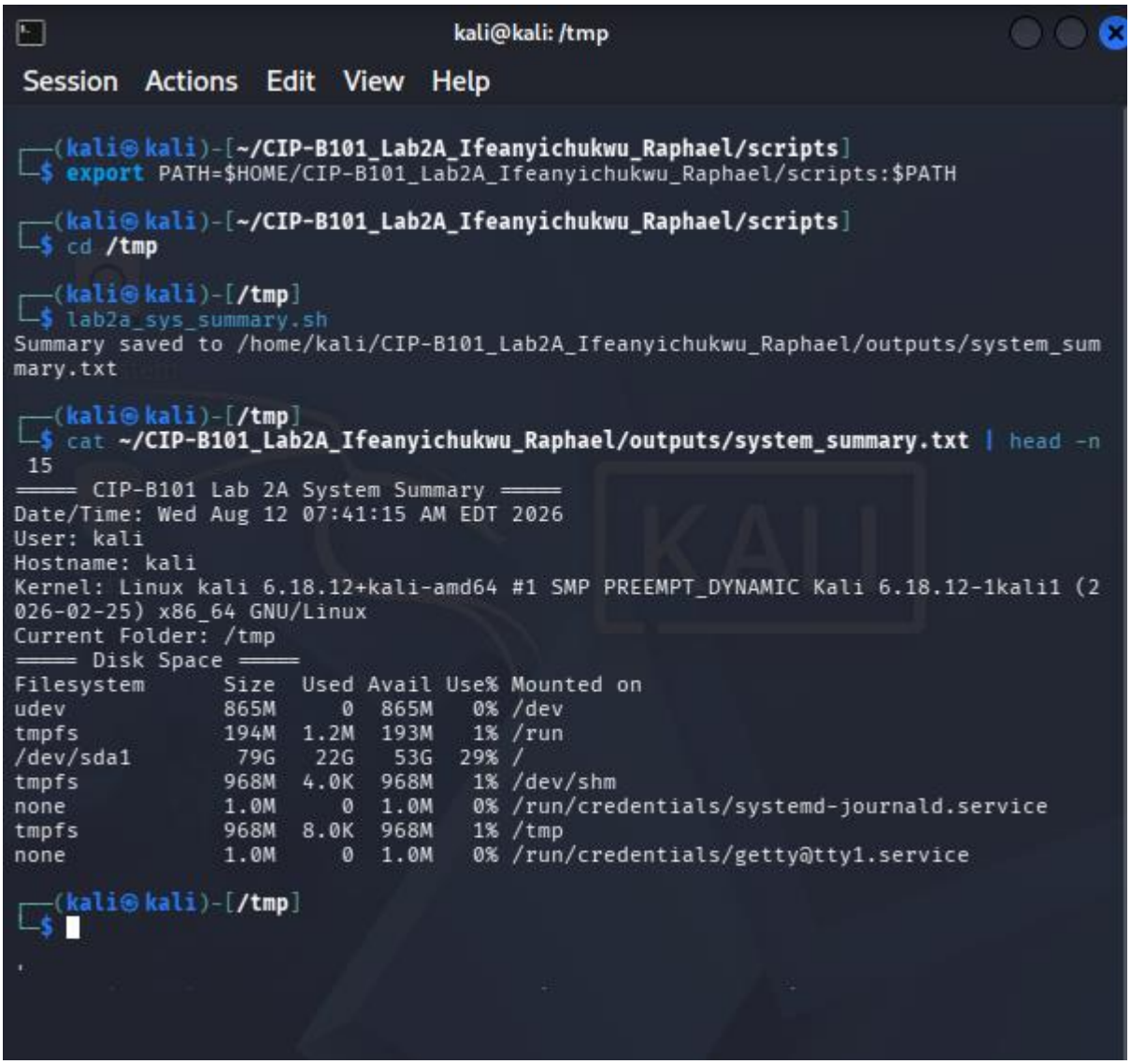
Testing against 127.0.0.1/localhost only ever queries the student's own machine, so there is no risk of probing, disrupting, or triggering an intrusion alert on a system that is not owned or authorized by the student. Scanning an external or unknown system, even briefly and even with tools intended for defensive use, can be indistinguishable from reconnaissance activity carried out ahead of an attack, and may violate computer misuse legislation or an organization's acceptable use policy regardless of intent. Restricting network commands to localhost in this lab keeps the exercise entirely within the student's own authorized environment while still demonstrating the same commands and syntax that would later be used, under proper authorization, against a real target.

Task 9 – Created and Executed a Simple Forensic Script



The script begins with a shebang line (#!/bin/sh) declaring which interpreter should run it, then defines an OUT variable pointing to the target report file. The curly braces group every echo and diagnostic command together so that their combined output can be redirected as a single block into $OUT, rather than needing a separate > redirection on every individual line. chmod +x grants the script execute permission, which is required before it can be run directly rather than only read as a text file.

Task 10 - Make the Script Callable from another Folder Using PATH

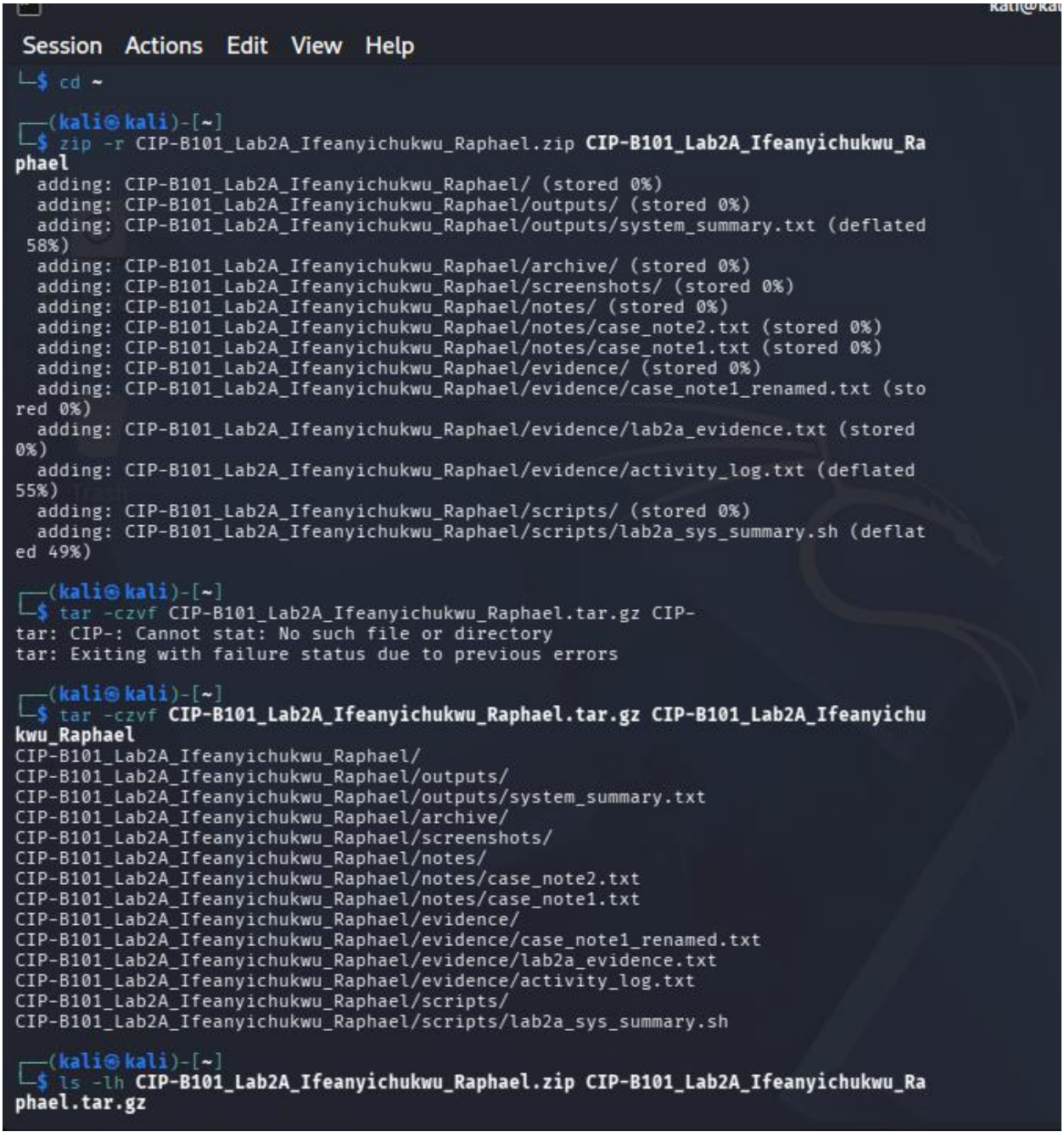


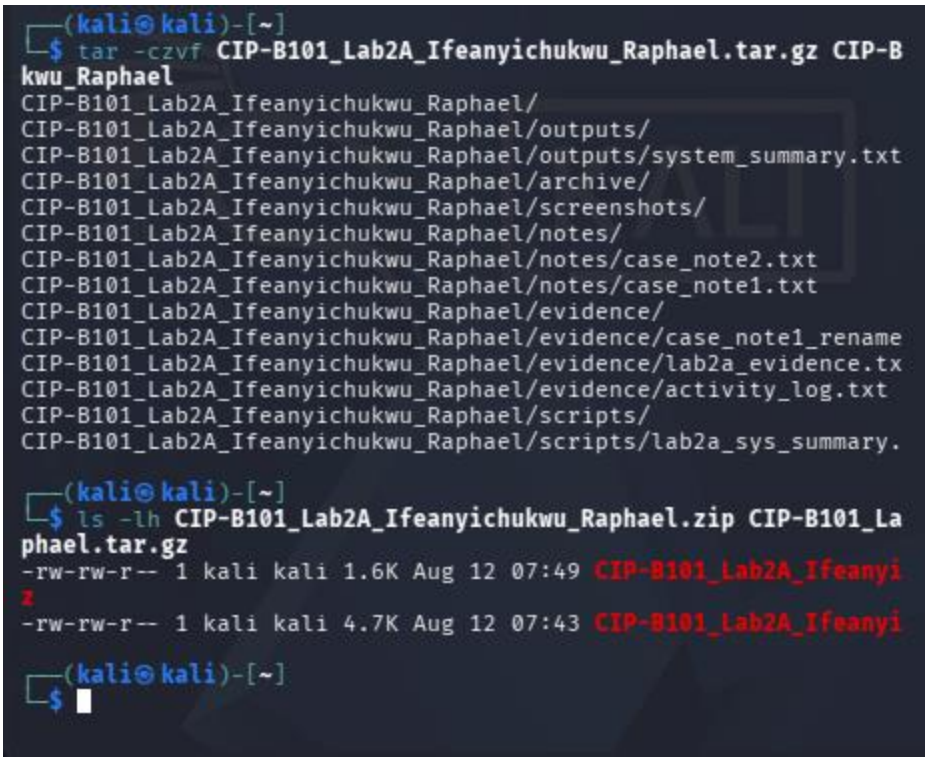
10.1 What PATH does

PATH is an environment variable holding an ordered, colon-separated list of directories that the shell searches whenever a command is typed without an explicit path in front of it. This is why a command such as ls can be run from any folder: ls itself lives in a directory such as /bin or /usr/bin, and because that directory is already included in PATH by default, the shell finds and executes it regardless of the user's current location. Prefixing the export command with the script's own scripts folder, followed by $PATH to preserve every existing entry,

temporarily adds that folder to the search list for the current terminal session, which is why lab2a_sys_summary.sh could then be run by name alone from /tmp without typing its full path.

Task 11 - Compress the Evidence Workspace





11.1 zip vs tar.gz in this context

zip -r compresses and archives the workspace in a single step, producing a .zip file that is widely readable across Windows, macOS and Linux without additional tools, which makes it a convenient choice for submission. tar -czvf instead first archives the folder using tar (bundling files together without compressing them) and then compresses that archive using gzip via the -z flag, producing a .tar.gz file that is the more conventional format on Linux systems and generally preserves Unix-specific file permissions and metadata more faithfully than zip does. Either format was acceptable for this lab, since the objective was to demonstrate the ability to package a completed workspace, not to compare compression efficiency.

Conclusion

Completing this lab reinforced that the Linux command line rewards precision, and that most of the real difficulty in this exercise came not from the underlying forensic concepts but from small, easily overlooked details in syntax and file naming. Early in the lab, a mismatch between the folder name typed in commands and the folder name actually created on disk (an extra space instead of an underscore) produced repeated "path not found" errors, which was only resolved by directly comparing command input against dir/ls output rather than assuming the names matched. A similar issue reappeared in the shell script written for Task 9, where leftover placeholder text in the OUT variable caused the script to run without error but silently fail to produce its output file, since it was writing to a folder that did not exist a useful reminder that a script exiting cleanly does not necessarily mean it did what was intended. Multi-line terminal pastes also proved unreliable throughout the lab, most notably when a long tar command was cut mid-argument, which was resolved by running commands individually rather than as a pasted block.

Beyond these practical lessons, the lab demonstrated that a complete evidence-handling workflow from workspace creation, through inode inspection, path navigation, grep-based log searching, and safe localhost-only network testing, to writing and relocating an executable script via PATH can be carried out entirely with tools already built into a standard Linux system. Packaging the finished workspace into both a .zip and a .tar.gz archive at the end tied the exercise together, producing a single, self-contained evidence package suitable for submission or handover. Overall, the lab underlined that careful verification at each step, rather than assuming a command has worked simply because it produced no visible error, is a habit worth carrying forward into more advanced forensic work.